

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Progettare il prompt

Tag e struttura

Perché i tag

Il prompt B del modulo, con le istruzioni da una parte e i dati dall'altra

Il prompt B del modulo: dal messaggio di un cliente, la Falegnameria Fittizia, una **scheda** per il responsabile del turno. Qui, in breve, le sue cinque righe stanno in `<istruzioni>` e il messaggio in `<messaggio>`

`il prompt B, in breve: […] segna un taglio`

```
<istruzioni>
Sei un operatore di secondo livello dell'assistenza clienti […]
Contesto: la scheda serve al responsabile del turno […]
Istruzione: estrai dal messaggio qui sotto i fatti rilevanti […]
Formato: una tabella con le colonne Problema | Quando | […]
Vincoli: […] ometti nome e numero di telefono del cliente.
</istruzioni>
<messaggio>
Buongiorno, vi scrivo per la Falegnameria Fittizia di Rovereto.
Venerdì mattina il centralino non funzionava […]
</messaggio>
```

I tag XML aiutano Claude a leggere **senza ambiguità** un prompt che mescola istruzioni, contesto, esempi e dati — soprattutto quando i dati sono tanti

Un tag per ogni tipo di contenuto (per esempio `<instructions>`, `<context>`, `<input>`): meno interpretazioni sbagliate — e il testo del cliente resta **un dato**, non un'istruzione

Regole dei tag

Nomi coerenti, tag annidati, e ogni testo al suo posto

- Nomi **descrittivi**, e **sempre gli stessi** in tutti i vostri prompt: se il messaggio del cliente sta in `<messaggio>`, ci sta in ogni prompt
- Tag **annidati** quando il contenuto ha una gerarchia: più documenti dentro `<documents>`, ciascuno nel suo `<document>`, con il testo in `<document_content>` e la provenienza in `<source>`
- Gli **esempi** in `<example>`, e più esempi in `<examples>`: così Claude li distingue dalle istruzioni — li riprende la parte «Esempi: quando aiutano, quando danneggiano»
- Il testo **incollato da altrove**, marcato come tale: nella sua guida per Opus, Anthropic suggerisce di distinguere il testo vostro da quello incollato, con un tag `<pasted_content>` — lo riprende il Modulo 7
- Anthropic per prima usa i tag: il system prompt di claude.ai che pubblica, quello di Sonnet, è diviso in sezioni come `<claude_behavior>` e `<product_information>`

Tag anche nella risposta

La struttura si chiede: i modelli recenti possono rispettarla in modo affidabile

- Chiedete che certe **sezioni della risposta** arrivino dentro tag dedicati: «scrivi la scheda dentro `<scheda>` e i dati mancanti dentro `<mancanti>`» — poi un programma ritaglia ogni sezione
- **La struttura si chiede**: i modelli recenti, scrive Anthropic, possono rispettare in modo affidabile anche strutture complesse, quando glielo si chiede; e per togliere i preamboli («Ecco la scheda…»), un'istruzione diretta nel system prompt
- Il **prefill** — scrivere voi l'inizio della risposta, che Claude continua — nell'API, sui modelli recenti, **non è più supportato**: la richiesta torna con un errore
- Per separare le sezioni, tag XML o titoli Markdown: la forma esatta, osserva Anthropic, conta probabilmente **sempre meno**

Solo JSON

M2-D03, 1 di 2: «SOLO un JSON», chiesto a parole, sui tre modelli

Indovinate, a voce: chiedo «rispondi SOLO con un JSON valido» — arriverà **pulito**? E il **telefono** del cliente ci sarà dentro?

`richiesta-json.md, con il messaggio accorciato: […]`

```
Estrai dal messaggio i campi seguenti e rispondi SOLO con un JSON
valido, senza testo prima o dopo, con esattamente queste chiavi:
tipo_cliente (privato, azienda o associazione), servizio,
problema, urgenza (bassa, media, alta o urgente),
azione_richiesta.

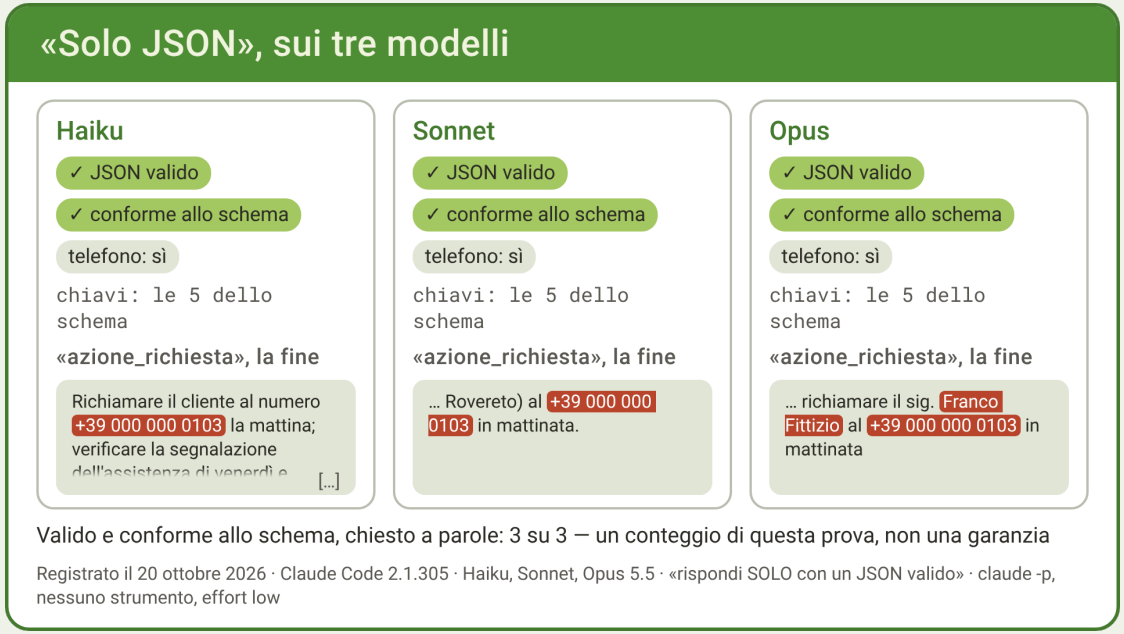
<messaggio>
Buongiorno, vi scrivo per la Falegnameria Fittizia di Rovereto.
[…] Mi potete richiamare al +39 000 000 0103, la mattina. […]
</messaggio>
```

Demo **M2-D03**, dal vivo: la richiesta a **Haiku**, **Sonnet** e **Opus**, una chiamata `claude -p` per modello, effort low, nessuno strumento — il comando è nel README dell'elemento, passo 2

Guardate: il testo si legge come **JSON**, così com'è, come lo leggerebbe un programma? Rispetta lo **schema** della demo — le cinque chiavi, nessuna in più, i valori ammessi? E dove finisce il **telefono**?

Solo JSON

M2-D03, 1 di 2: «SOLO un JSON», chiesto a parole, sui tre modelli



Haiku, Sonnet e Opus, una chiamata ciascuno a effort low: il testo letto come lo legge un programma, e la fine del campo azione_richiesta (i puntini segnano il taglio)
Valido e conforme allo schema, chiesto a parole: un conteggio, non una garanzia
© Simone Scannapieco — figura dalla registrazione del 20 ottobre 2026

- Anthropic, «Output strutturati»: «Senza output strutturati, Claude può generare risposte JSON malformate», anche con un buon prompt; e «Best practice per il prompting»: «i modelli più recenti possono rispettare in modo affidabile schemi complessi quando viene loro richiesto»
- Il controllo della demo, nella registrazione: **3 su 3** validi e conformi allo schema — un conteggio di questa prova, **non una garanzia**
- Il telefono del cliente è in `azione_richiesta` in **3 risposte su 3**: nessuno aveva detto che cosa fare dei dati personali

La forma garantita

M2-D03, 2 di 2: la stessa richiesta, con uno schema

- Il formato si può anche **imporre**: con `--output-format json` e `--json-schema`, `claude -p` restituisce un JSON **convalidato** su uno schema, nel campo `structured_output` — solo in modalità non interattiva

```
claude -p "$(cat richiesta-json.md)" --model haiku --effort low \
  --tools "" --system-prompt "$(cat istruzione.txt)" \
  --strict-mcp-config --no-session-persistence \
  --output-format json --json-schema "$(cat schema.json)" \
  > risposte/schema-haiku.json
python3 leggi_json.py structured_output \
  < risposte/schema-haiku.json
# passo 4: lo stesso comando con schema-non-valido.json, lo STDERR
# e il codice d'uscita in risposte/schema-non-valido.txt
```

- `schema.json`: le cinque chiavi, **tutte obbligatorie**, nessuna in più, e per `tipo_cliente` e `urgenza` i valori ammessi; `schema-non-valido.json`: lo stesso con una parola cambiata, e **non è** uno JSON Schema valido
- Guardate il campo `structured_output`, con l'oggetto convalidato — e il telefono? Con lo schema non valido: il **codice d'uscita** e la riga d'errore
- Per chi scrive script, e per chi li fa girare sui sistemi: l'oggetto si legge da quel campo, senza ripulire il testo — dopo aver controllato che il campo ci sia

La forma garantita

M2-D03, 2 di 2: la stessa richiesta, con uno schema

`structured_output` di Haiku, registrato: «…» segna un taglio

```
{"tipo_cliente": "azienda", "servizio": "Centralino telefonico …",
 "problema": "Venerdì mattina (circa dalle 8 alle 10:30-11) …",
 "urgenza": "alta",
 "azione_richiesta": "… Franco Fittizio al +39 000 000 0103 …"}
```

`schema-non-valido.json`: lo STDERR registrato, «…» un taglio

```
Error: --json-schema is not a valid JSON Schema: data/type must …
```

- **Garantito** (Anthropic, «Eseguire Claude Code a livello programmatico»): con `--json-schema` l'oggetto, conforme allo schema, è in `structured_output`, o il campo manca — nella registrazione **confermata**: il campo c'è, conforme
- **Garantito**, nella stessa pagina: uno schema non valido ferma `claude` all'avvio — nella registrazione **confermata**: uscita 1, nessuna risposta
- **Non garantito: il contenuto** — lo schema toglie gli errori di sintassi, **non quelli di significato** (Anthropic, guida all'esame Claude Certified Architect). Una **dimostrazione**, senza verdetto: il controllo cambia `tipo_cliente`, «privato» al posto di «azienda», e la copia è ancora conforme. E il **telefono** è nell'oggetto

Il punto

Tag e schema: che cosa fa ciascuno

- **I tag**: le istruzioni da una parte, i dati dall'altra — un tag per tipo di contenuto, nomi coerenti, anche nella risposta
- **Lo schema**: quando la risposta finisce in un programma, `--json-schema` ne garantisce la forma; che cosa ci finisce dentro — i dati personali, i valori che mancano — si decide nel prompt e nello schema
- Nell'esercitazione «Il prompt pessimo», il **giudice** che misura i vostri prompt risponde così: uno schema, e i suoi verdetti in `structured_output`

I tag separano le istruzioni dai dati; lo schema garantisce la forma, non il contenuto.

Subito dopo: gli esempi, quando aiutano e quando danneggiano.